FastAPI

The whole map, in plain language, with the code you actually write.

FastAPI is a Python library for building an API. An API is a menu of requests a program accepts. A phone app, a web page, or another server sends a request. Your FastAPI program checks it, does the work, and sends a response back. This book is that menu, from the first “hello” through a database, login, tests, and a tidy folder layout.

You need the Python from the Python guide: functions, types, dictionaries, `async`, and classes. You do not need to already know the web.

Contents

1. A request and a response
2. The smallest app
3. Path, query, and body
4. Pydantic, the shape checker
5. Status codes and errors
6. Many paths, one project
7. Dependencies
8. A database
9. The same tasks in MongoDB
10. Login
11. The edges: files, CORS, background work, sockets
12. Tests and settings
13. A day of building one resource
14. Interview questions and answers

1. A request and a response

Picture a restaurant. The customer is the **client**. The waiter carries one **request**: which dish, and any notes. The kitchen is your function. The plate that comes back is the **response**.

On the web, the request is an HTTP message.

- The **method** is the kind of job. `GET` reads. `POST` creates. `PUT` replaces. `PATCH` changes a few fields. `DELETE` removes.
- The **path** is the dish name, such as `/tasks/3`.
- The **query** is a note after a question mark: `/tasks?done=true`.
- The **body** is a parcel of JSON, usually on create and update.
- The **headers** are labels on the envelope, such as the login token.

The response has a status code and usually a JSON body. `200` means success. `201` means created. `404` means not found. `401` means you have not proved who you are. `422` means the parcel failed the shape check. FastAPI sends `422` for you when a type or a rule fails.

JSON is text that looks like Python dictionaries and lists. FastAPI turns a Python dict or a Pydantic model into JSON on the way out, and turns JSON into a Python object on the way in.

2. The smallest app

Install the server and the runner. From a virtual environment:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install fastapi uvicorn
```

A file named `main.py`:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "hello"}
```

`app` is the restaurant. `@app.get("/")` says “when a GET arrives for `/`, call `home`.” The decorator is the menu entry. The function is the kitchen. A dict return value becomes JSON.

Start it:

```
uvicorn main:app --reload
```

`main` is the file. `app` is the variable. `--reload` restarts when you save, which is for your machine, not for a real server. Open `http://127.0.0.1:8000`. Open `http://127.0.0.1:8000/docs`. That second page is Swagger, built from your type hints. You can try every route in the browser. `/redoc` is the same menu in a reading layout.

Use `def` for a normal function. FastAPI runs it in a worker thread so a slow database call does not freeze other requests. Use `async def` when you `await` a library that is actually async. Do not put a slow blocking call, such as `time.sleep` or a normal database driver, inside `async def`. That freezes every request on the loop. If you are unsure, `def` is the safe start.

3. Path, query, and body

FastAPI looks at the function arguments and decides where each value comes from.

```
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/tasks/{task_id}")
def read_task(task_id: int, q: str | None = None):
    return {"task_id": task_id, "q": q}
```

`task_id` is in the path because the path string contains `{task_id}`. The hint `int` means a request for `/tasks/hello` is rejected with 422. You never write `int(task_id)` yourself. `q` is not in the path and is not a body model, so it is a query parameter. `/tasks/3?q=milk` fills both.

A singular Pydantic model (next chapter) is read from the JSON body. If you need two body models, or you want to be obvious, wrap the argument in `Body()`. Headers use `Header()`. Cookies use `Cookie()`.

```
from typing import Annotated
from fastapi import Header, Path, Query

@app.get("/tasks/{task_id}")
def read_task(
    task_id: Annotated[int, Path(ge=1)],
    limit: Annotated[int, Query(ge=1, le=100)] = 10,
    x_request_id: Annotated[str | None, Header()] = None,
):
    return {"task_id": task_id, "limit": limit, "request_id": x_request_id}
```

`Annotated` is the modern way to say “this type, plus these rules.” `ge` means greater than or equal. `le` means less than or equal. Header names like `x_request_id` become `X-Request-Id` on the wire.

Put fixed paths before paths with parameters when they can clash. `/tasks/count` declared after `/tasks/{task_id}` can be swallowed by the parameter route. Declare `/tasks/count` first.

4. Pydantic, the shape checker

A Pydantic model is a class that describes a JSON object. FastAPI uses it to check the request and to build the docs. If a field is missing or the wrong type, the client gets 422 and your function does not run.

```
from pydantic import BaseModel, Field

class TaskIn(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    done: bool = False

class TaskOut(BaseModel):
    id: int
    title: str
    done: bool

@app.post("/tasks", response_model=TaskOut, status_code=201)
def create_task(task: TaskIn):
    return {"id": 1, "title": task.title, "done": task.done, "secret": "nope"}
```

`response_model` is the plate you promise. The extra key `secret` is dropped from the JSON. That is how you keep a password hash out of a user response: leave it off `UserOut`. The client sends `TaskIn`. You return something that can fill `TaskOut`.

Defaults belong on the model. `done: bool = False` means a missing `done` becomes false. A field without a default is required.

Nested models work. A list field is `tags: list[str] = []`. Be careful with a mutable default on a normal function. On a Pydantic model, a default of `[]` is safe in current Pydantic. Prefer `Field(default_factory=list)` when you want to be obvious.

You can add a check that looks at more than one field:

```
from pydantic import model_validator

class RangeIn(BaseModel):
    start: int
    end: int

    @model_validator(mode="after")
    def end_after_start(self):
        if self.end < self.start:
            raise ValueError("end must be >= start")
        return self
```

5. Status codes and errors

Set the success code on the decorator. Raise `HTTPException` when the job cannot finish for a reason the client should understand.

```
from fastapi import HTTPException

TASKS = {1: {"id": 1, "title": "Milk", "done": False}}

@app.get("/tasks/{task_id}", response_model=TaskOut)
def read_task(task_id: int):
    task = TASKS.get(task_id)
    if task is None:
        raise HTTPException(status_code=404, detail="Task not found")
    return task
```

`detail` becomes the JSON message. Use 404 for a missing row, 409 for a duplicate name, 400 for a rule that is not a shape error, 401 for a bad or missing login, 403 when you know who they are and they may not do this.

A handler can catch your own exception type and turn it into a response, so services raise `NotFound` and the web layer translates it.

```
from fastapi import Request
from fastapi.responses import JSONResponse

class NotFound(Exception):
    pass

@app.exception_handler(NotFound)
def not_found(_request: Request, exc: NotFound):
    return JSONResponse(status_code=404, content={"detail": "Not found"})
```

Do not catch every exception and hide it. An unexpected bug should surface as a 500 in the logs. The client gets a generic message. You get the traceback.

6. Many paths, one project

One file is fine for a lesson. A real app splits by resource. An `APIRouter` is a mini app you include into the main app. The prefix is glued onto every path. Tags group the docs page.

```
from fastapi import APIRouter

router = APIRouter(prefix="/tasks", tags=["tasks"])

@router.get("")
def list_tasks():
    return []

# main.py
from tasks import router as tasks_router
app.include_router(tasks_router)
```

A layout that stays readable:

```
app/
  main.py          # create FastAPI, include routers, lifespan
  database.py      # engine and session
  models.py        # SQLAlchemy tables
  schemas.py       # Pydantic in and out
  routers/
    tasks.py
  deps.py          # shared Depends functions
  security.py
```

`main.py` should stay boring: build the app, attach routers, attach middleware. Rules about tasks live in the tasks router and in functions it calls. When a function needs the database, it receives a session. It does not construct the engine itself.

7. Dependencies

`Depends` is FastAPI calling a helper for you and passing the result in. The helper can check a header, open a database session, or load the current user. If the helper raises `HTTPException`, your path function never runs.

```
from typing import Annotated
from fastapi import Depends

def get_page(limit: int = 10, offset: int = 0):
    return {"limit": limit, "offset": offset}

Page = Annotated[dict, Depends(get_page)]

@app.get("/tasks")
def list_tasks(page: Page):
    return page
```

A dependency can depend on another dependency. The current user depends on the token. The token depends on the header. You write that chain once.

A dependency that must clean up uses `yield`. The code before `yield` runs on the way in. The code after runs after the response, even when the path raised.

```
def get_session():
    session = SessionLocal()
    try:
        yield session
    finally:
        session.close()
```

That is the database session pattern. Every request gets its own session. The session closes when the request ends. You do not share one session across requests.

8. A database

The dictionary in chapter 5 disappears when the process restarts. A database keeps the rows. SQLite is a file, perfect for learning. PostgreSQL is the usual server later. The Python code is almost the same. This chapter uses SQLAlchemy, the library that turns rows into objects.

```
python -m pip install sqlalchemy
```

```
from sqlalchemy import create_engine, String, Boolean
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, sessionmaker

class Base(DeclarativeBase):
    pass

class Task(Base):
    __tablename__ = "tasks"
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    done: Mapped[bool] = mapped_column(Boolean, default=False)

engine = create_engine("sqlite:///./tasks.db")
SessionLocal = sessionmaker(bind=engine)
Base.metadata.create_all(engine)
```

`create_all` makes missing tables. It does not change a table you already created when you add a column. Real projects use a migration tool, Alembic, for those changes. For the first day, `create_all` is enough.

Create, read, update, delete, with the session from `Depends`:

```
from sqlalchemy.orm import Session
from fastapi import Depends

SessionDep = Annotated[Session, Depends(get_session)]

@router.post("", response_model=TaskOut, status_code=201)
def create_task(payload: TaskIn, db: SessionDep):
    row = Task(title=payload.title, done=payload.done)
    db.add(row)
    db.commit()
    db.refresh(row)
    return row

@router.get("/{task_id}", response_model=TaskOut)
def read_task(task_id: int, db: SessionDep):
    row = db.get(Task, task_id)
    if row is None:
        raise HTTPException(status_code=404, detail="Task not found")
    return row

@router.patch("/{task_id}", response_model=TaskOut)
def update_task(task_id: int, payload: TaskIn, db: SessionDep):
    row = db.get(Task, task_id)
    if row is None:
        raise HTTPException(status_code=404, detail="Task not found")
    row.title = payload.title
    row.done = payload.done
    db.commit()
    db.refresh(row)
    return row

@router.delete("/{task_id}", status_code=204)
def delete_task(task_id: int, db: SessionDep):
    row = db.get(Task, task_id)
    if row is None:
        raise HTTPException(status_code=404, detail="Task not found")
    db.delete(row)
    db.commit()
```

`commit` saves. `refresh` reloads the row so `id`, which the database just assigned, is filled in. `204` means success with an empty body. A list endpoint is `db.query(Task).offset(offset).limit(limit).all()`, or the newer `db.scalars(select(Task)).all()`. Always limit a list. An unbounded “return every row” becomes the outage.

The Pydantic `TaskOut` can be filled from the SQLAlchemy object if you set `model_config = ConfigDict(from_attributes=True)` on the model. Then returning `row` works with `response_model=TaskOut`.

8a. The same tasks in MongoDB

SQLite stores rows in tables. MongoDB stores documents in collections. A document is one JSON-like object. A collection is the pile of those objects, such as all tasks. There are no joins to write. You put the fields you want to read together on the same document.

Install MongoDB Community on your machine, start it, and install the Python driver. The default address is `mongodb://localhost:27017`.

```
python -m pip install pymongo
```

The driver is synchronous, so the path functions stay `def`. FastAPI runs them in a thread pool. One client is created at startup and closed at shutdown. Each request borrows the collection. The client itself is safe to share.

```
from pymongo import MongoClient
from pymongo.collection import Collection

client = MongoClient("mongodb://localhost:27017")
tasks: Collection = client.taskapp.tasks

def get_tasks() -> Collection:
    return tasks
```

MongoDB assigns `_id` as an `ObjectId`. JSON does not understand that type, so the response turns it into a string. The create body is still `TaskIn`. The response is still `TaskOut`.

```
from typing import Annotated
from bson import ObjectId
from bson.errors import InvalidId
from fastapi import APIRouter, Depends, HTTPException

router = APIRouter(prefix="/tasks", tags=["tasks"])
Tasks = Annotated[Collection, Depends(get_tasks)]

def to_out(doc: dict) -> dict:
    return {"id": str(doc["_id"]), "title": doc["title"], "done": doc["done"]}

@router.post("", response_model=TaskOut, status_code=201)
def create_task(payload: TaskIn, tasks: Tasks):
    result = tasks.insert_one({"title": payload.title, "done": payload.done})
    doc = tasks.find_one({"_id": result.inserted_id})
    return to_out(doc)

@router.get("/{task_id}", response_model=TaskOut)
def read_task(task_id: str, tasks: Tasks):
    try:
        oid = ObjectId(task_id)
    except InvalidId:
        raise HTTPException(status_code=404, detail="Task not found")
    doc = tasks.find_one({"_id": oid})
    if doc is None:
        raise HTTPException(status_code=404, detail="Task not found")
    return to_out(doc)

@router.get("")
def list_tasks(tasks: Tasks, limit: int = 20):
    docs = tasks.find().limit(limit)
    return [to_out(doc) for doc in docs]

@router.delete("/{task_id}", status_code=204)
def delete_task(task_id: str, tasks: Tasks):
    try:
        oid = ObjectId(task_id)
    except InvalidId:
        raise HTTPException(status_code=404, detail="Task not found")
    result = tasks.delete_one({"_id": oid})
    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail="Task not found")
```

Declare `GET /tasks` before `GET /tasks/{task_id}` if you ever use a fixed word such as `/tasks/count`. A bad id string is not a crash. `ObjectId` rejects it, and you answer 404.

Update replaces the fields you name and leaves the rest. `$set` is that partial write.

```
tasks.update_one({"_id": oid}, {"$set": {"done": True}})
```

You do not call `commit`. The write is finished when `insert_one` or `update_one` returns. There is no session to close per request unless you open a transaction, which this lesson does not need.

Pick one database for the task app while you learn. SQLite when you want rows, foreign keys, and the SQLAlchemy chapter. MongoDB when each task is a document and you want to add a field without a migration. The routes, the Pydantic models, and the status codes stay the same. Only the four lines that save and load change.

	SQLite with SQLAlchemy	MongoDB
A record	A row in `tasks`	A document in the `tasks` collection
Id	Integer, assigned by the table	`ObjectId`, sent to the client as a string
Save	`add`, then `commit`	`insert_one`
Load one	`db.get(Task, id)`	`find_one({"_id": oid})`
Change	Set attributes, then `commit`	`update_one` with `$set`

9. Login

A public create-task route is a lesson. A real route asks who is calling. The usual FastAPI shape is OAuth2's password flow plus a bearer token. The words are heavier than the idea.

1. The client sends a username and password to `POST /token`.
2. You check the password against a stored hash. You never store the raw password. A hash made for passwords, such as bcrypt, is one-way.
3. You return a signed token that says "this is user 4, and this token dies at this time." A JWT is that signed parcel.
4. Later calls send `Authorization: Bearer <token>`.
5. A dependency reads the header, checks the signature and the expiry, loads the user, or raises 401.

```
python -m pip install "pyjwt[crypto]"
```

```
from datetime import datetime, timedelta, timezone
import jwt
from fastapi import Depends
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm

oauth2 = OAuth2PasswordBearer(tokenUrl="token")
SECRET = "change-me"  # real apps read this from the environment

def make_token(user_id: int) -> str:
    payload = {
        "sub": str(user_id),
        "exp": datetime.now(timezone.utc) + timedelta(minutes=30),
    }
    return jwt.encode(payload, SECRET, algorithm="HS256")

def get_current_user(token: Annotated[str, Depends(oauth2)]):
    try:
        payload = jwt.decode(token, SECRET, algorithms=["HS256"])
    except jwt.PyJWTError:
        raise HTTPException(status_code=401, detail="Invalid token")
    return {"id": int(payload["sub"])}

@app.post("/token")
def login(form: Annotated[OAuth2PasswordRequestForm, Depends()]):
    # look up form.username, check form.password against the hash
    return {"access_token": make_token(1), "token_type": "bearer"}

@app.get("/me")
def me(user: Annotated[dict, Depends(get_current_user)]):
    return user
```

The password check is the part you must not fake in production. Hash on signup. On login, hash the attempt with the same algorithm and compare. If you compare the raw password to a column of raw passwords, a stolen database is a stolen set of passwords.

Put `Depends(get_current_user)` on routes that need a person. Leave `/token` and `/health` open. The docs page gets an Authorize button because of `OAuth2PasswordBearer`, so you can try protected routes in `/docs`.

A token in a JWT cannot be unread until it expires, unless you keep a denylist. Short expiry, thirty minutes here, limits the damage of a stolen token. A refresh token is a second, longer token used only to get a new

access token. Add it when the product needs to stay logged in for days. The first version can ask the user to log in again.

10. The edges

CORS

A browser page on `http://localhost:5173` calling an API on `http://127.0.0.1:8000` is a different origin. The browser blocks it unless the API says that origin is allowed. That permission is CORS. It is a browser rule. A phone app or `curl` does not care.

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

List the real front-end origins. `allow_origins=["*"]` together with credentials is rejected by browsers. During a local lesson, the Vite origin above is the one you want.

Files and forms

JSON is the body for structured data. A file upload is multipart form data.

```
from fastapi import File, UploadFile

@app.post("/photos")
async def upload(file: UploadFile = File(...)):
    data = await file.read()
    return {"name": file.filename, "bytes": len(data)}
```

For a large file, read it in chunks and write it to disk or to object storage. Do not hold a video in memory. A normal HTML form uses `Form()` fields instead of a JSON model. JSON and form fields do not mix in one body.

Background work

Some work should not make the client wait: sending mail, writing a log line. `BackgroundTasks` runs after the response is sent, in the same process.

```
from fastapi import BackgroundTasks

def write_log(task_id: int):
    print("created", task_id)

@app.post("/tasks", status_code=201)
def create_task(payload: TaskIn, background: BackgroundTasks):
    background.add_task(write_log, 1)
    return {"id": 1, "title": payload.title}
```

If the process dies, that task dies with it. Mail you must not lose belongs in a real queue, the queue from the system-design book. Background tasks are for small, acceptable-to-lose side work.

WebSockets

HTTP is one request, one response. A WebSocket stays open so the server can push. Chat and live counts use it.

```
from fastapi import WebSocket

@app.websocket("/ws")
async def ws(websocket: WebSocket):
    await websocket.accept()
    while True:
        text = await websocket.receive_text()
        await websocket.send_text(f"echo {text}")
```

The loop ends when the client disconnects. Catch `WebSocketDisconnect` and remove that client from whatever list you were broadcasting to. This is the same “connection registry” idea as the chat design. FastAPI gives you the socket. You still decide who receives which message.

Lifespan

Code that should run once when the process starts, and once when it stops, goes in the lifespan. Opening a connection pool belongs here. Creating tables for a lesson can live here too.

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(engine)
    yield
    engine.dispose()

app = FastAPI(lifespan=lifespan)
```

The code before `yield` is startup. The code after is shutdown.

11. Tests and settings

A test calls the app without starting uvicorn. `TestClient` pretends to be a browser.

```
python -m pip install httpx
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_home():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "hello"}

def test_missing_task():
    response = client.get("/tasks/999")
    assert response.status_code == 404
```

Override a dependency so tests do not touch the real database or the real login. The test passes a fake.

```
def fake_user():
    return {"id": 1}

app.dependency_overrides[get_current_user] = fake_user
```

Clear overrides at the end of the test, or later tests inherit the fake. A database test uses a separate SQLite file, or an in-memory engine, created in a fixture and thrown away after. Do not point tests at the database your evening project is using.

What to test first: the happy create, the 404, the 422 from a blank title, and one protected route without a token, which must be 401. That is enough to trust the menu. You are not testing FastAPI. You are testing your rules.

Settings

Secrets and machine-specific values stay out of the code. A `.env` file holds them locally and is listed in `.gitignore`. The process reads them from the environment.

```
python -m pip install pydantic-settings
```

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env")
    secret_key: str
    database_url: str = "sqlite:///./tasks.db"

settings = Settings()
```

A `.env` file:

```
SECRET_KEY=change-me
DATABASE_URL=sqlite:///./tasks.db
```

Names are matched without caring about case. If `SECRET_KEY` is missing, the app refuses to start. That is what you want. A default secret that ships in git is how a token gets forged.

12. A day of building one resource

Build tasks, in this order. Each step runs before you add the next. If you add login and the database and files on the same afternoon, you will not know which piece failed.

1. A dict and four routes: list, create, read, delete. Open `/docs` and click them.
2. Replace the dict argument with `TaskIn` and `response_model=TaskOut`. Send a blank title and watch 422.
3. Raise 404 for a missing id. Test it with `TestClient`.
4. Move the routes onto a router and include it from `main.py`.
5. Swap the dict for SQLite. The session comes from `Depends`.
6. Add `/token` and put `get_current_user` on create and delete. List can stay public for the lesson.
7. Add CORS only when a browser page on another port calls you.

A request walks this line. Client, router, dependencies, your function, database, response model, JSON. When something breaks, point at the step. 422 is the shape, before your function. 404 is your function. 401 is the user dependency. 500 is an exception you did not translate, so read the traceback.

The slips

- **Blocking call inside `async def`.** Use `def`, or an async database driver and `await`.
- **One global session.** Create a session per request and close it in `finally`.
- **Returning the SQLAlchemy row with a secret column and no `response_model`.** The whole object can leak. Set the response model.
- **Password stored as text.** Store a password hash.
- **Secret key in the file you commit.** Read it from the environment.
- **`/tasks/{task_id}` declared above `/tasks/count`.** The word `count` is captured as an id. Fixed paths go first.
- **Forgetting `commit`.** The row exists only in the session and vanishes.
- **CORS added to please `curl`.** `curl` was never blocked. Look at the browser console. The origin must match exactly, including the port.
- **A test that uses the real `tasks.db`.** Override the session dependency.

13. Interview questions and answers

What is FastAPI? A Python web framework that turns type hints into request checking, JSON responses, and automatic docs. A path function is one menu entry.

What is the difference between path, query, and body? The path is part of the URL and identifies the resource, `/tasks/3`. The query is the filter after `?`. The body is the JSON parcel, described by a Pydantic model.

Why 422? The request reached the app and failed validation: wrong type, missing field, or a field rule. Your function did not run. 400 is for a rule you enforce yourself inside the function.

What does `response_model` do? It filters and documents the output. Extra fields on the object you return are dropped. That is how a password hash stays off the wire.

`def` or `async def`? `def` runs in a thread pool and may call blocking libraries. `async def` runs on the event loop and must `await` instead of blocking. A blocking call inside `async def` stalls other requests.

What is `Depends`? A function FastAPI runs for you and injects into the path. Use it for the database session, paging, and the current user. `yield` dependencies get a cleanup step after the response.

How does login work? `POST /token` checks a password against a hash and returns a signed JWT. Later requests send `Authorization: Bearer`. A dependency decodes the token and raises 401 when the signature or the expiry fails.

Why hash passwords? A stolen database of hashes is not a list of passwords. You store a slow password hash, bcrypt or Argon2, and check new attempts against it. You cannot read the original password back, and you should not be able to.

What is CORS? A browser refusing to let a page on one origin read a response from another origin, unless the API allows that origin. Tools that are not browsers skip this check.

How do you test a route? `TestClient` calls the app in process. Override dependencies so the test supplies a fake user and a throwaway database. Assert the status code and the JSON.

Where do settings live? In the environment, read by a settings object. The secret key is not a string in git.

SQLite or MongoDB? SQLite, through SQLAlchemy, stores rows in tables and gives you integer ids and `commit`. MongoDB stores documents in a collection. The id is an `ObjectId` you send as a string, and `insert_one` writes immediately. The routes and the Pydantic models stay the same. Only the load and save lines change. Use one of them while you learn the menu.

When do you not use BackgroundTasks? When the work must survive a crash. Use a real queue. Background tasks run after the response, in the same process, and die with it.

You have the map

A FastAPI app is a menu of functions. Types and Pydantic check the request. The response model checks what leaves. Dependencies hand in the session and the user. The database remembers the rows. Login is a password hash plus a short-lived token. Tests call the menu without the network. Everything else, files, CORS, sockets, and background work, is a specific kind of request or a specific kind of after-work.

Build the task resource in the order in chapter 12. When a status code appears, you already know which step produced it.

End of the FastAPI guide.